



SESSION 9

INTEGRATION OF NODE.JS WITH MONGODB

Learning Objectives

In this session, students will learn to:

- Describe the process to connect Node.js with MongoDB
- Demonstrate the CRUD operations on a MongoDB database using Node.js
- Explain the building of CRUD APIs in Node.js

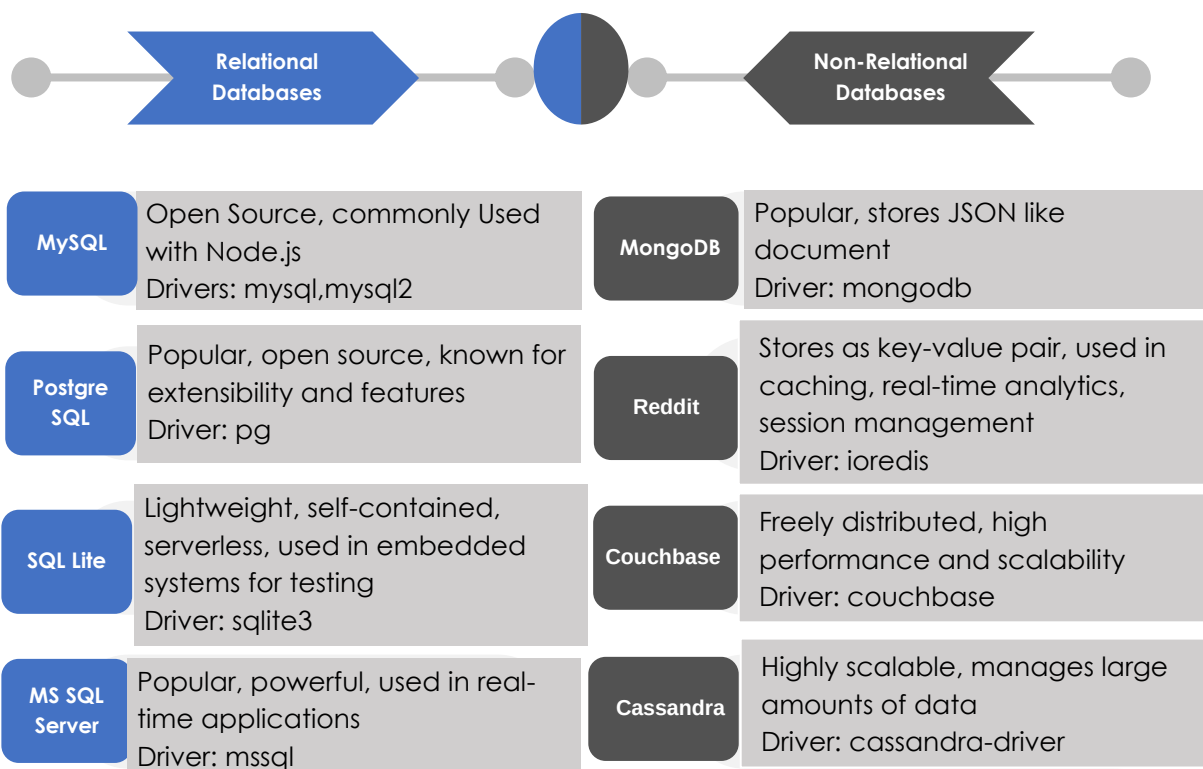
Database is an important part of any software application. It provides the space for storing the information or data. The arrangement of data in the database is subject to the requirements of the system. Once the data is arranged, various operations can be carried out on the data. To perform these operations, an interaction is essential between the data and the application. This session will explain steps to install MongoDB database and make it interact with Node.js to perform required tasks.

9.1 Introduction to Database Connectivity

Data is an integral part of any application that must be collected, stored, and updated periodically for future reference. To facilitate data management in a Web application there must be interaction between the database layer and the presentation layer. This interaction is enabled through a component

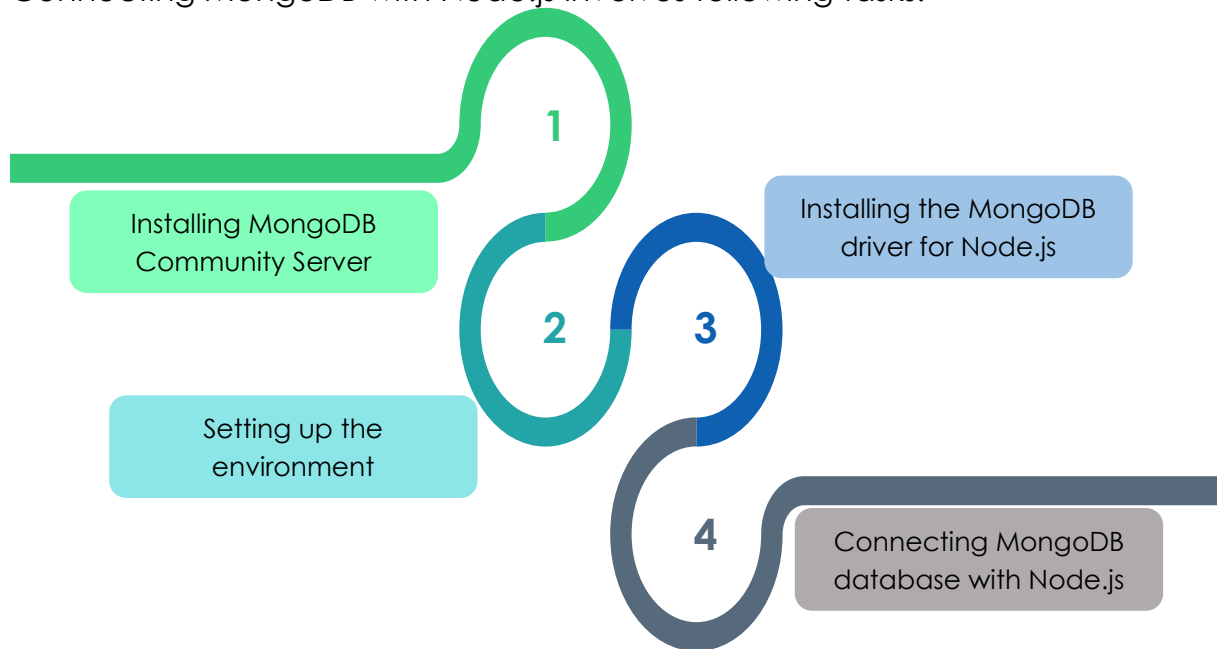
referred to as a driver, which acts as an intermediate communication point between the database and the presentation part or the front end.

Node.js is a runtime environment that supports a wide range of relational and NoSQL databases. It facilitates the interaction between the database layer and the presentation layer. It has a variety of libraries and modules that make the interaction with the databases easy. These libraries provide the methods and functions for storing, updating, and retrieving data. Developers can select a database system based on the application requirements and install the required database drivers using the Node Package Manager (npm). Some of the common databases supported by Node.js are as follows:



9.2 Connecting MongoDB with Node.js

Connecting MongoDB with Node.js involves following tasks:



Installing MongoDB Community Server

To install the MongoDB Community Server on Windows systems, perform following steps:

1. Open a browser and navigate to the URL:
<https://www.mongodb.com/try/download/community>
The MongoDB Community Server Download page opens.
2. Scroll down and click **Select Package** under the **Community Server** section.
3. On this page, scroll down and ensure that:
 - In the **Version** drop-down, **7.0.3 (current)** is selected.
 - In the **Platform** drop-down, **Windows x64** is selected.
 - In the **Package** drop-down, **msi** is selected.
4. Click **Download**.
5. After the download is complete, run the installer.

The **MongoDB 7.0.3 2008R2Plus SSL (64 bit) Setup** wizard opens as shown in Figure 9.1.

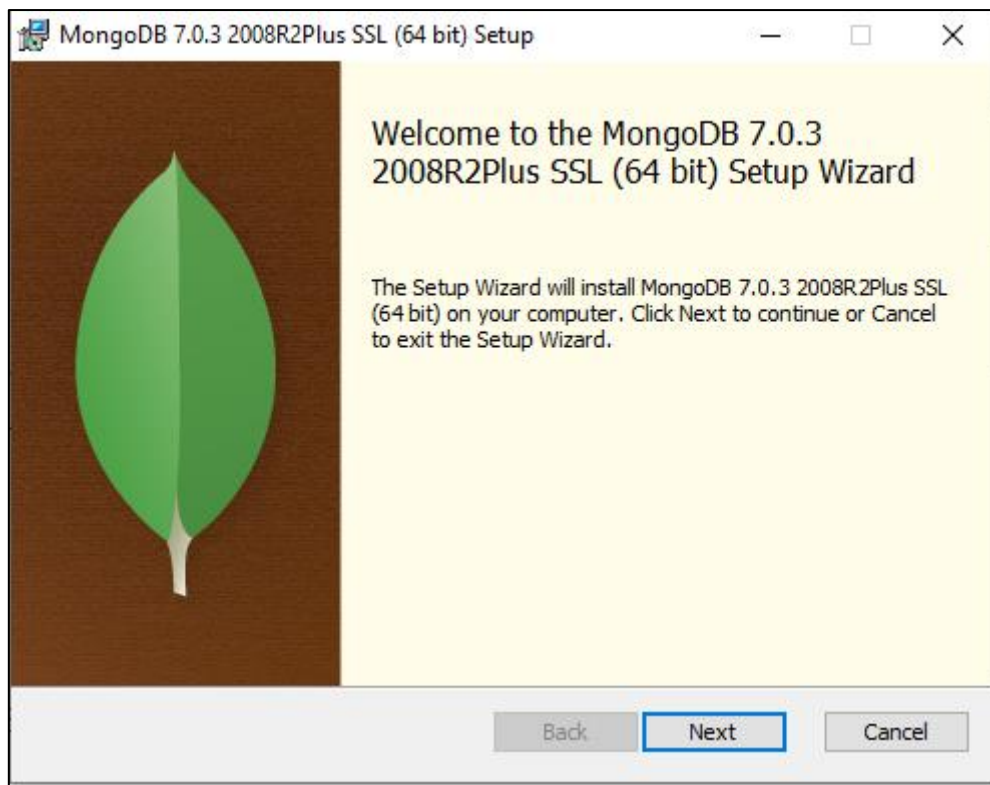


Figure 9.1: Welcome Page of the MongoDB Installation Wizard

6. Click **Next**.

The **End-User License Agreement** page of the wizard opens as shown in Figure 9.2.

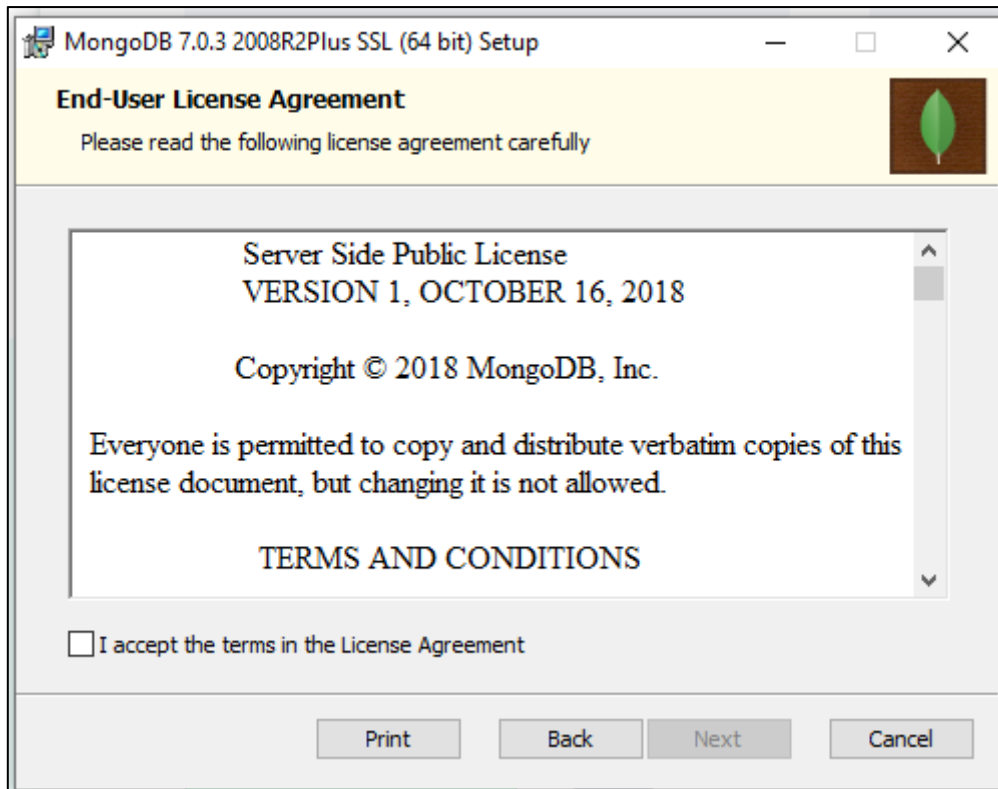


Figure 9.2: End-User License Agreement Page

7. To proceed, select the **I accept the terms in the License Agreement** check box.
8. Click **Next**.

The **Choose Setup Type** page of the wizard opens as shown in Figure 9.3.

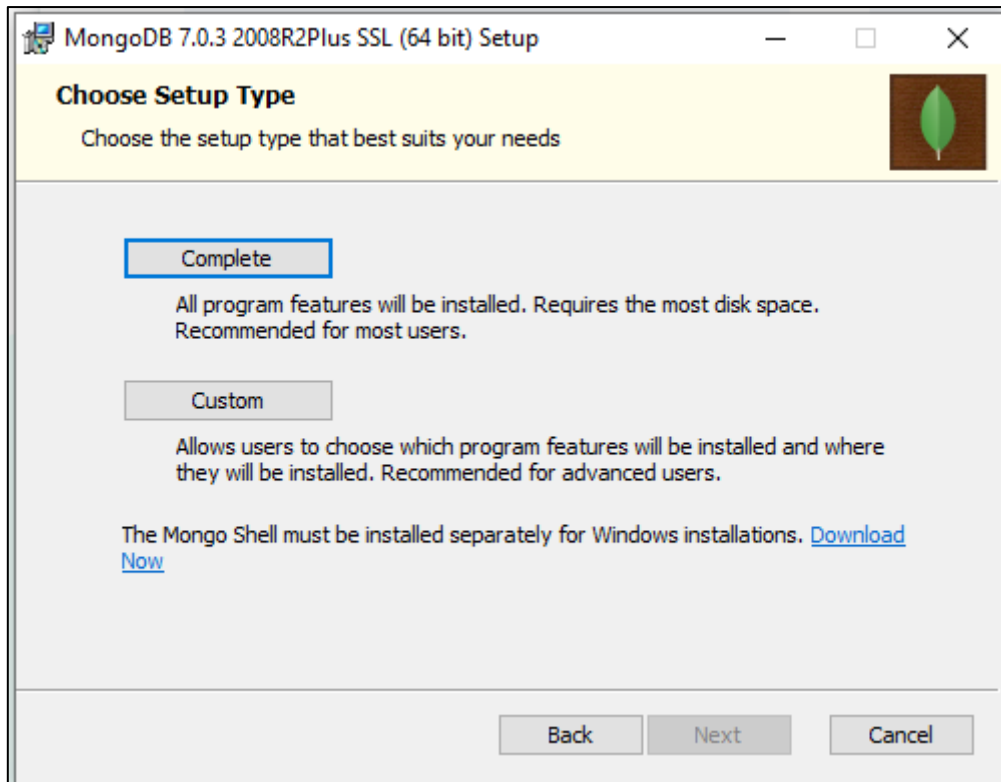


Figure 9.3: Choose Setup Type Page

9. Click **Complete**.

The **Service Configuration** page of the wizard opens as shown in Figure 9.4.

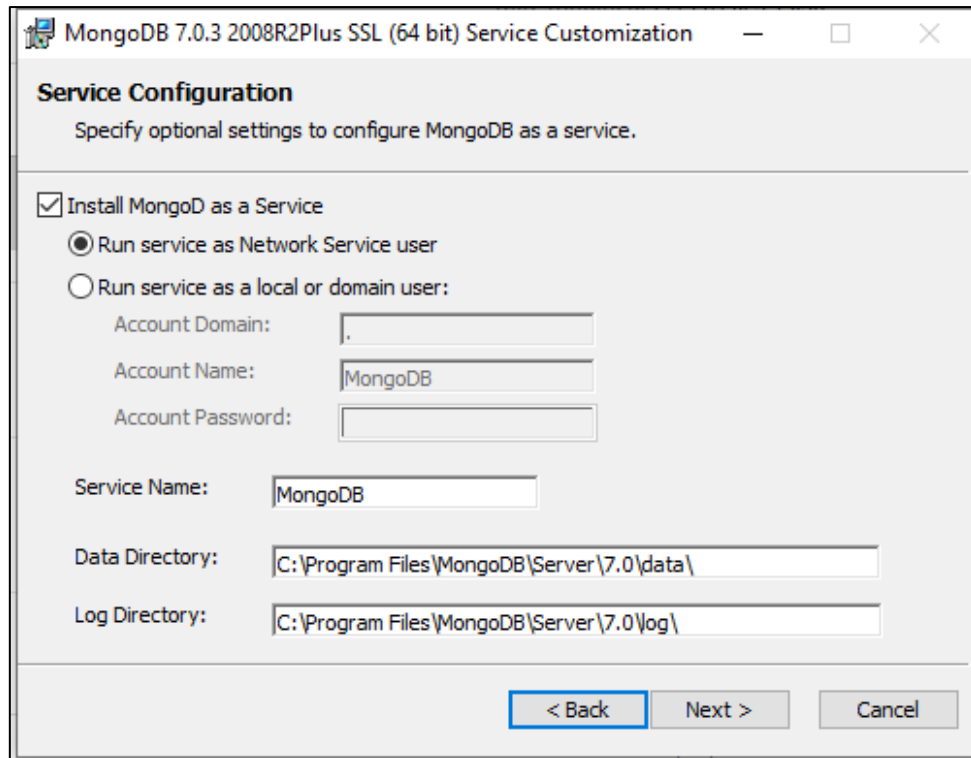


Figure 9.4: Service Configuration Page

10. Click **Next**.

The **Install MongoDB Compass** page of the wizard opens as shown in Figure 9.5.

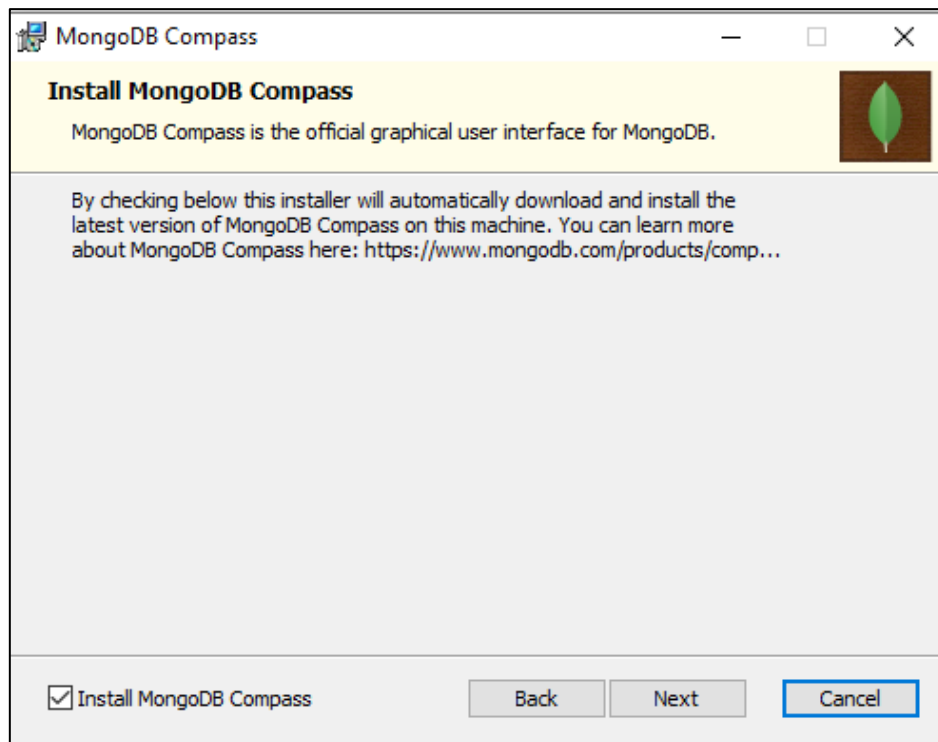


Figure 9.5: Install MongoDB Compass Page

11. Ensure that the **Install MongoDB Compass** checkbox is selected.

12. Click **Next**.

The **Ready to install MongoDB 7.0.3 2008R2Plus SSL (64 bit)** page of the wizard opens as shown in Figure 9.6.

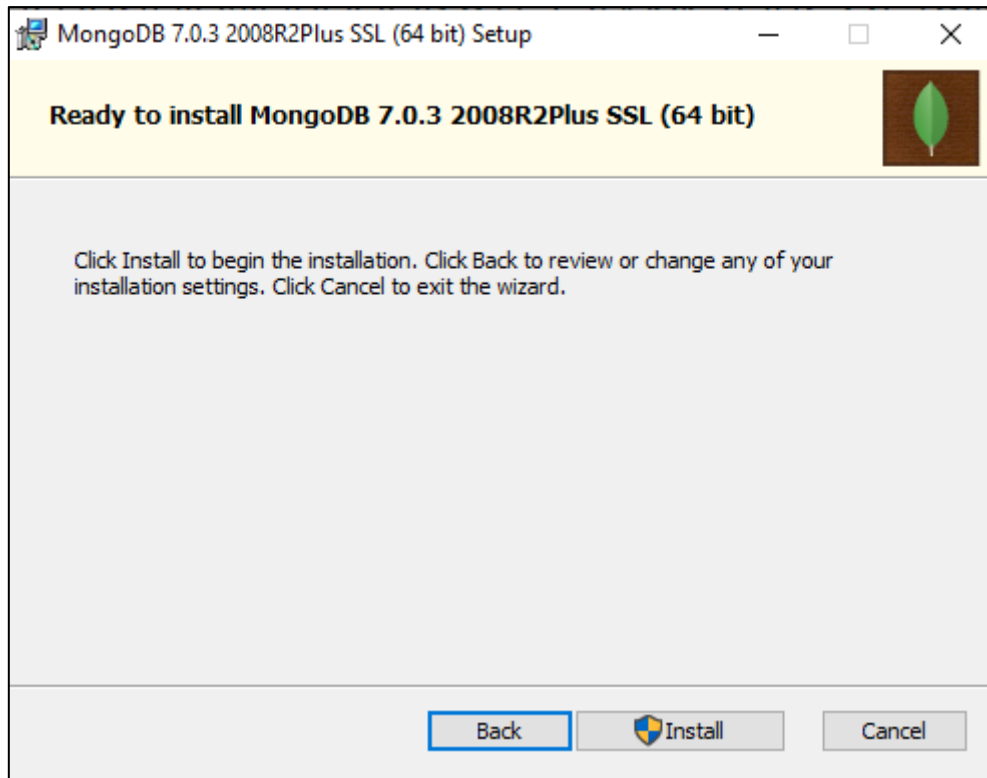


Figure 9.6: Ready to Install Page

13. Click **Install**.

The **Installing MongoDB 7.0.3f 2008R2Plus SSL (64 bit)** page of the wizard opens as shown in Figure 9.7.

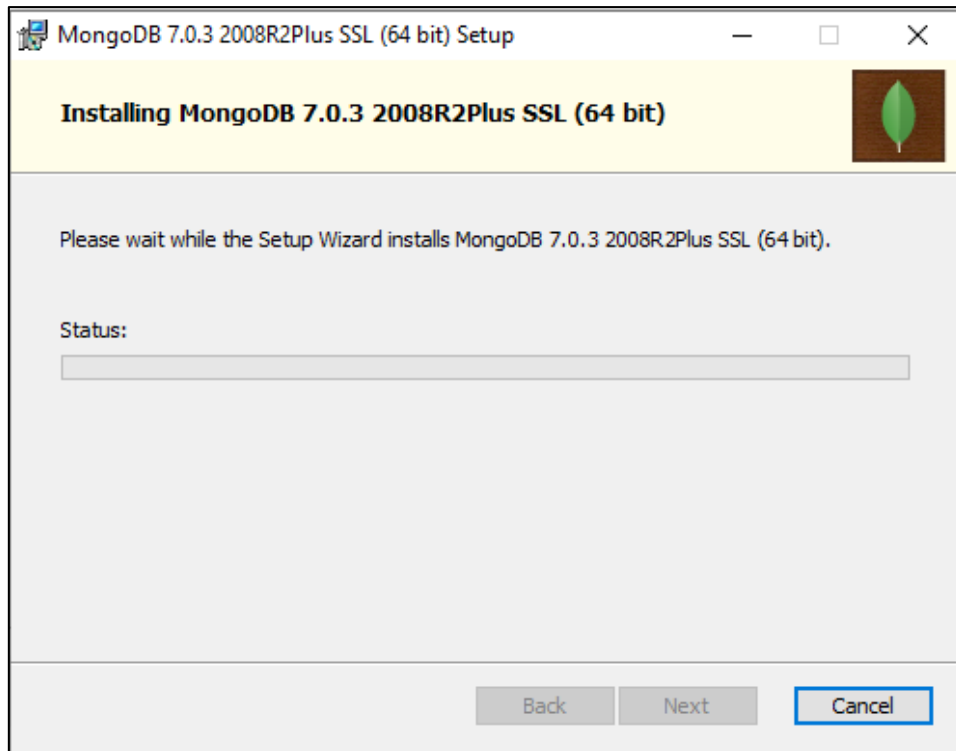


Figure 9.7: Installation in Progress Page

The **Completed the MongoDB 7.0.3 2008R2Plus SSL (64 bit) Setup Wizard** page opens as shown in Figure 9.8.

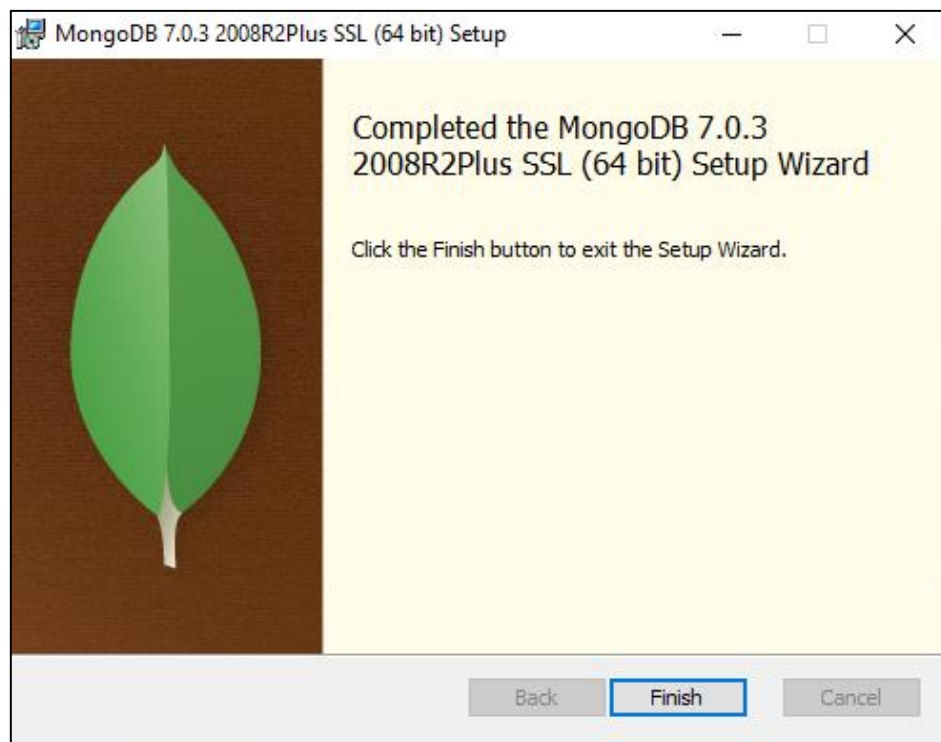


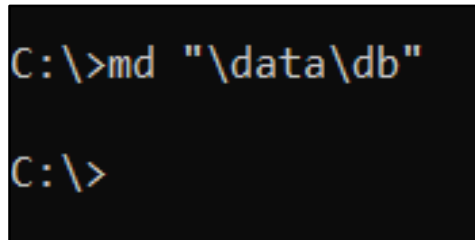
Figure 9.8: Installation Completion Page

14. Click **Finish**.

Setting up the Environment

To set up the environment, perform following steps:

1. Open Windows Command Prompt (cmd.exe) as an **Administrator**.
2. Create the data directory where MongoDB will store all the data, as shown in Figure 9.9.



```
C:\>md "\data\db"
C:\>
```

Figure 9.9: Creating Directories

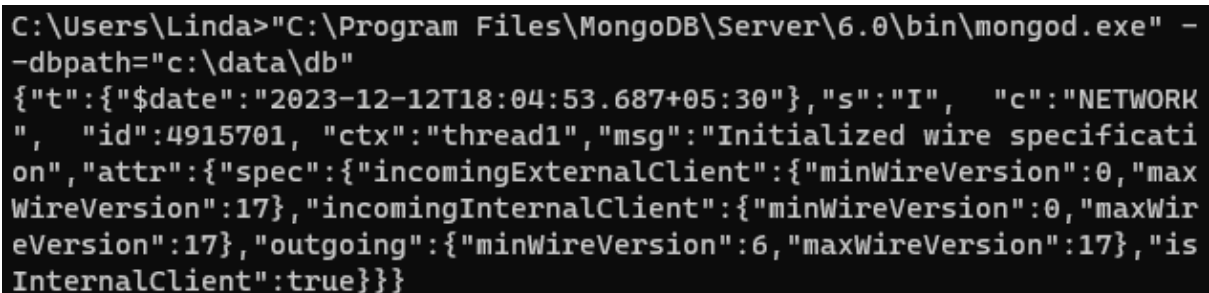


MongoDB listens for connections from clients only on port 27017, by default. The default storage path for data is /data/db directory.

To start the **MongoDB** database, run the command:

```
"C:\Program Files\MongoDB\Server\6.0\bin\mongod.exe" --
dbpath="c:\data\db"
```

The command executes as shown in Figure 9.10.



```
C:\Users\Linda>"C:\Program Files\MongoDB\Server\6.0\bin\mongod.exe" -
--dbpath="c:\data\db"
{"t":{"$date":"2023-12-12T18:04:53.687+05:30"},"s":"I", "c":"NETWORK
", "id":4915701, "ctx":"thread1","msg":"Initialized wire specificati
on","attr":{"spec":{"incomingExternalClient":{"minWireVersion":0,"max
WireVersion":17},"incomingInternalClient":{"minWireVersion":0,"maxWir
eVersion":17},"outgoing":{"minWireVersion":6,"maxWireVersion":17},"is
InternalClient":true}}}
```

Figure 9.10: Starting the MongoDB Database

The `--dbpath` option specifies the database directory. If the MongoDB database server is running correctly, the message: **waiting for connections** is displayed as shown in Figure 9.11.

```
{
  "t": { "$date": "2023-12-12T18:04:57.015+05:30", "s": "I", "c": "FTDC",
    "id": 20631, "ctx": "ftdc", "msg": "Unclean full-time diagnostic data capture shutdown detected, found interim file, some metrics may have been lost", "attr": { "error": { "code": 0, "codeName": "OK" } } }
}, {
  "t": { "$date": "2023-12-12T18:04:57.034+05:30", "s": "I", "c": "NETWORK",
    "id": 23015, "ctx": "listener", "msg": "Listening on", "attr": { "address": "127.0.0.1" } }
}, {
  "t": { "$date": "2023-12-12T18:04:57.034+05:30", "s": "I", "c": "NETWORK",
    "id": 23016, "ctx": "listener", "msg": "Waiting for connections", "attr": { "port": 27017, "ssl": "off" } }
}
```

Figure 9.11: Waiting for Connections Message

Now, the MongoDB instance `mongod` is successfully running.



Add `C:\Program Files\MongoDB\Server\6.0\bin` to the system path. This allows executing the `mongod` instance instantly without having to navigate to this folder each time to run the instance.

Installing the MongoDB Driver for Node.js

After installing the MongoDB server and setting up the necessary environment, the driver for Node.js to access the MongoDB database must be installed.

To install native MongoDB drivers using npm, run the command:

```
npm install mongodb
```

The command executes as shown in Figure 9.12.

```
C:\Users\Linda\nodejs>npm install mongodb

added 12 packages, and audited 13 packages in 3s

found 0 vulnerabilities
```

Figure 9.12: Installation of MongoDB Driver

Connecting MongoDB Database with Node.js

After installing the driver, connect the MongoDB database with Node.js.

To connect to a MongoDB database, perform following steps:

1. Open VS Code or Notepad and type the code as shown in Code Snippet 1.

Code Snippet 1:

```
const { MongoClient } = require('mongodb')
const client = new MongoClient('mongodb://localhost:27017')
client.connect()
  .then(() => {
    console.log('Connected to MongoDB Successfully!')
    console.log('Exiting from MongoDB')
    client.close()
  })
  .catch(error => console.log('Failed to connect to MongoDB!', error))
```

2. Save this code as `connectmongo.js`.

Table 9.1 describes Code Snippet 1.

Code	Description
<code>const { MongoClient } = require('mongodb')</code>	Imports the <code>MongoClient</code> class from the <code>MongoDB</code> package to initiate the connection with a <code>MongoDB</code> database using the <code>MongoDB Node.js driver</code>
<code>const client = new MongoClient('mongodb://localhost:27017')</code>	Creates a new <code>MongoDB</code> client instance by specifying the connection URL
<code>client.connect()</code>	Initiates the connection to the <code>MongoDB</code> database and returns a <code>promise</code> object
<code>.then(() => { console.log() })</code>	Displays success messages to exhibit that the connection has been established successfully
<code>client.close()</code>	Closes the connection to the <code>MongoDB</code> database and frees up the resources
<code>.catch(error => console.log())</code>	Displays any errors while establishing the connection

Table 9.1: Description of Code Snippet 1

To run `connectmongo.js`, type following command in the Command Prompt:

```
node connectmongo.js
```

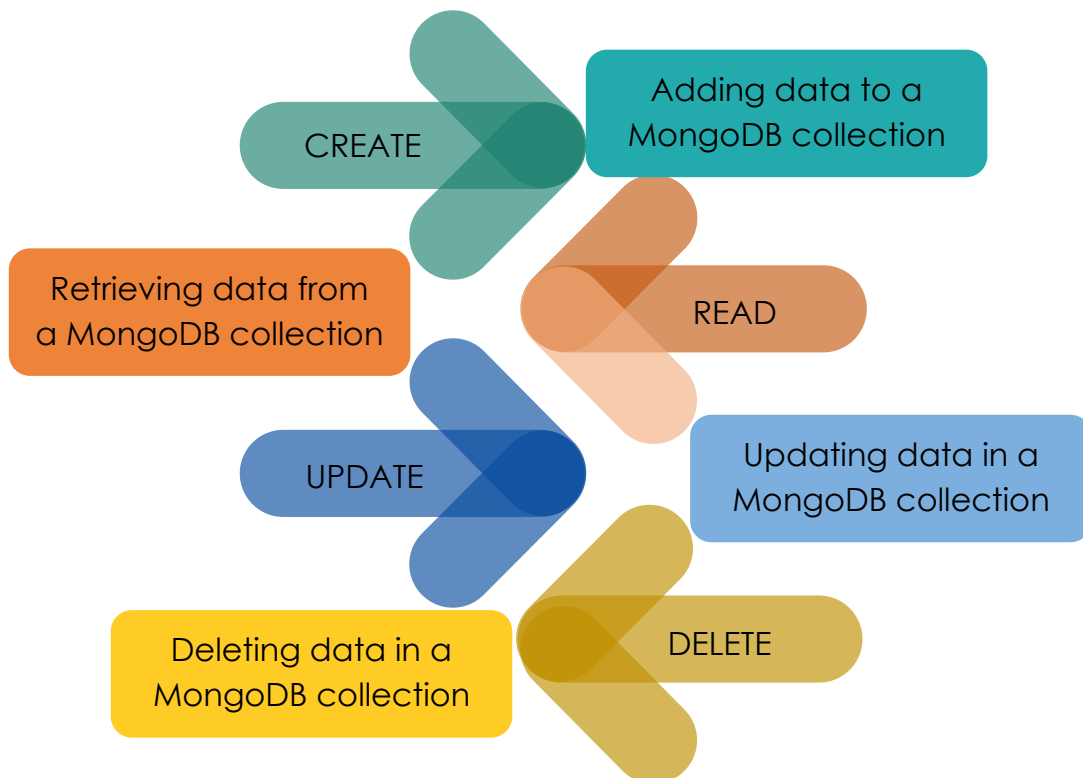
Once the server is up and running, a message is logged to the console to indicate that the connection to `MongoDB` is successful, as shown in Figure 9.13.

```
C:\Users\Linda\nodejs>node connectmongo.js  
Connected to MongoDB Successfully!  
Exiting from MongoDB
```

Figure 9.13: Console Displaying Successful Connection to MongoDB

9.3 Performing Create, Read, Update, and Delete (CRUD) Operations in Node.js

Developers can perform CRUD operations to manage data in a database.



In a MongoDB database, data is organized into collections and each document in a collection represents a data record.

9.3.1 Create Operation

Let us create a MongoDB database named, `Library`. This database will contain the `books` collection with three documents.

1. Open VS Code or Notepad and type the code given in Code Snippet 2.

Code Snippet 2:

```
const { MongoClient } = require('mongodb');  
const url = 'mongodb://localhost:27017';  
const databaseName = 'Library';
```

```

const collectionName = 'books';

const dataToInsert = [
  { Book_id:101, Book_name: 'The Secret of The House On The Hill', Author_name: 'Julia Harris', Price: '$10', Age_group:'15+', Book_type:'Thriller' },
  { Book_id:2001, Book_name: 'The Court Of The Evil', Author_name: 'Nolan Evans', Price: '$17', Age_group:'10+',Book_type:'Young-Adult' },
  { Book_id:3001, Book_name: 'When You Wish Upon A Star', Author_name: 'Ashlyn Clark', Price: '$18.99', Age_group:'18+',Book_type:'Romance' },
];

const client = new MongoClient(url);
client.connect()
  .then(() => {
    const db = client.db(databaseName);
    const collection = db.collection(collectionName);
    return collection.insertMany(dataToInsert);
  })
  .then(result => {
    console.log(`${result.insertedCount} documents inserted into the collection in the "${databaseName}" database.`);
  })
  .catch(err => {
    console.error('Failed to connect to Database:', err);
  })
  .finally(() => {
    client.close();
  });

```

2. Save the code as createop.js.

The code:

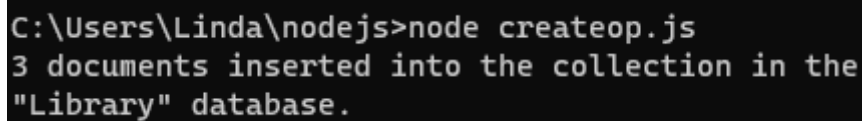
- Imports the `MongoClient` class and defines the connection details, names of the database and collection, and an array of documents to insert into the collection.
- Creates a new `MongoClient` instance and establishes a connection to the MongoDB server specified in the URL using the `connect` method.
- Accesses the database and collection using the success callback argument in the `then` block.
- Creates a reference to the database and collection using the `client.db(databaseName)` and `db.collection(collectionName)`.
- Inserts the `dataToInsert` array into the collection using the `insertMany` method.
- Handles the results to log the number of documents that were successfully inserted using another `then` block.

- Handles the errors thrown during connecting or inserting using the `catch` block.
- Ensures that the MongoDB client connection is closed using the `finally` block.

To execute the code, type following command in the Command Prompt:

```
node createop.js
```

A message is logged in the console to indicate that three documents are inserted into the `books` collection of the `Library` database, as shown in Figure 9.14.



```
C:\Users\Linda\nodejs>node createop.js
3 documents inserted into the collection in the
"Library" database.
```

Figure 9.14: Console Displaying Insertion of Documents into the Collection

9.3.2 Read Operation

Let us view the three documents inserted into the `books` collection of the `Library` database.

1. Open VS Code or Notepad and type the code given in Code Snippet 3.

Code Snippet 3:

```
const { MongoClient } = require('mongodb');
const url = 'mongodb://localhost:27017';
const databaseName = 'Library';
const collectionName = 'books';
const client = new MongoClient(url);
client.connect()
  .then(() => {
    const db = client.db(databaseName);
    const collection = db.collection(collectionName);
    return collection.find({}).toArray();
  })
  .then(docs => {
    console.log(`Found ${docs.length} documents in the
"${collectionName}" collection:`);
    console.log(docs);
  })
  .catch(err => {
    console.error('Failed to connect to Database:', err);
  })
```

```
.finally(() => {  
    client.close();  
});
```

2. Save the code as `readop.js`.

The code:

- Imports the `MongoClient` class and defines the connection details, names of the database and collection, and an array of documents.
- Creates a new `MongoClient` instance and establishes a connection to the MongoDB server specified in the URL using the `connect` method.
- Accesses the `Library` database and the `books` collection using the success callback in the `then` block.
- Retrieves all the documents in the collection using the `find` method and converts the result to an array of documents using the `toArray` method.
- Handles the retrieved documents and logs them to the console using another `then` block.
- Manages any errors thrown during connecting or querying using the `catch` block.
- Ensures that the MongoDB client connection is closed using the `finally` block.

To execute the code, type following command in the Command Prompt:

```
node readop.js
```

Find the three documents retrieved from the `books` collections of the `Library` database, as shown in Figure 9.15.


```

C:\Users\Linda\nodejs>node readop.js
Found 3 documents in the "books" collection:
[
  {
    _id: new ObjectId('65785deb18ee8801b5712490'),
    Book_id: 101,
    Book_name: 'The Secret of The House On The Hill',
    Author_name: 'Julia Harris',
    Price: '$10',
    Age_group: '15+',
    Book_type: 'Thriller'
  },
  {
    _id: new ObjectId('65785deb18ee8801b5712491'),
    Book_id: 2001,
    Book_name: 'The Court Of The Evil',
    Author_name: 'Nolan Evans',
    Price: '$17',
    Age_group: '10+',
    Book_type: 'Young-Adult'
  },
  {
    _id: new ObjectId('65785deb18ee8801b5712492'),
    Book_id: 3001,
    Book_name: 'When You Wish Upon A Star',
    Author_name: 'Ashlyn Clark',
    Price: '$18.99',
    Age_group: '18+',
    Book_type: 'Romance'
  }
]

```

Figure 9.15: Console Displaying the Retrieved Documents

9.3.3 Update Operation

Let us update Price and Age_group values in the document having Book_id as 2001. The new Price value should be \$20 and the new Age_group value should be 12+.

1. Open VS Code or Notepad and type the code given in Code Snippet 4.

Code Snippet 4:

```

const { MongoClient } = require('mongodb');
const url = 'mongodb://localhost:27017';

```

```

const databaseName = 'Library';
const collectionName = 'books';
const client = new MongoClient(url);
client.connect()
  .then(() => {
    const db = client.db(databaseName);
    const collection = db.collection(collectionName);
    // Define the filter criteria to match the document to
update
    const filter = { Book_id:2001};
    const update = {
      $set: {
        Price: '$20',
        Age_group: '12+',
      },
    };
    return collection.updateOne(filter, update);
  })
  .then(result => {
    if (result.matchedCount === 0) {
      console.log('No matching document found for the
update.');
```

2. Save the code as updateop.js.

The code:

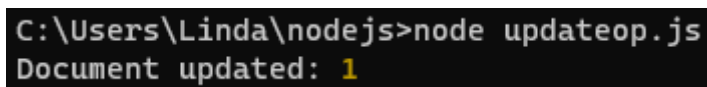
- Imports the `MongoClient` class and defines the connection details, names of the database and collection, and an array of documents.
- Creates a new `MongoClient` instance and establishes a connection to the MongoDB server specified in the URL, using the `connect` method.
- Accesses the `Library` database and the `books` collection, using the success callback in the `then` block.
- Defines the criteria to identify the document to be modified using the `filter` object.
- Specifies the modifications to be applied using the `update` object and the `$set` operator.
- Updates the document that matches the filter criteria, using the `updateOne` method.

- Inserts a result callback to check if a matching document is found for updating and to log the number of documents modified.
- Manages any errors thrown during connecting using the `catch` block.
- Ensures that the MongoDB client connection is closed using the `finally` block.

To execute the code, type following command in the Command Prompt:

```
node updateop.js
```

Find the number of documents that are updated in the `books` collections of the `Library` database as shown in Figure 9.16.



```
C:\Users\Linda\nodejs>node updateop.js
Document updated: 1
```

Figure 9.16: Console Displaying the Number of Updated Documents

To verify if the document with `Book_id` as `2001` has been updated, retrieve all the documents in the `books` collection by executing `readop.js`.

Type following command in the Command Prompt:

```
node readop.js
```

The `Price` and `Age_group` values have been updated in the specified document as shown in Figure 9.17.

```

C:\Users\Linda\nodejs>node readop.js
Found 3 documents in the "books" collection:
[
  {
    _id: new ObjectId('65785deb18ee8801b5712490'),
    Book_id: 101,
    Book_name: 'The Secret of The House On The Hill',
    Author_name: 'Julia Harris',
    Price: '$10',
    Age_group: '15+',
    Book_type: 'Thriller'
  },
  {
    _id: new ObjectId('65785deb18ee8801b5712491'),
    Book_id: 2001,
    Book_name: 'The Court Of The Evil',
    Author_name: 'Nolan Evans',
    Price: '$20',
    Age_group: '12+',
    Book_type: 'Young-Adult'
  },
  {
    _id: new ObjectId('65785deb18ee8801b5712492'),
    Book_id: 3001,
    Book_name: 'When You Wish Upon A Star',
    Author_name: 'Ashlyn Clark',
    Price: '$18.99',
    Age_group: '18+',
    Book_type: 'Romance'
  }
]

```

Figure 9.17: Console Displaying the Updated Values in the Document

9.3.4 Delete Operation

Let us delete the document having Book_id:2001.

1. Open VS Code or Notepad and type the code given in Code Snippet 5.

Code Snippet 5:

```

const { MongoClient } = require('mongodb');
const url = 'mongodb://localhost:27017';
const databaseName = 'Library';
const collectionName = 'books';
const client = new MongoClient(url);

```

```

client.connect()
  .then(() => {
    const db = client.db(databaseName);
    const collection = db.collection(collectionName);
    const filter = { Book_id: 2001 };
    return collection.deleteOne(filter);
  })
  .then(result => {
    if (result.deletedCount === 0) {
      console.log("Book not found");
    } else {
      console.log("Book deleted successfully")
    }
  })
  .catch(err => {
    console.error('Failed to connect to Database:', err);
  })
  .finally(() => {
    client.close();
  });

```

2. Save the code as deleteop.js.

The code:

- Imports the `MongoClient` class and defines the connection details, names of the database and collection, and an array of documents.
- Creates a new `MongoClient` instance and establishes a connection to the MongoDB server specified in the URL, using the `connect` method.
- Accesses the `Library` database and the `books` collection, using the success callback in the `then` block.
- Defines the criteria to identify the document to be deleted, using the `filter` object.
- Deletes the document that matches the filter criteria, using the `deleteOne` method.
- Inserts a result callback to check if a matching document is found for deleting and to log the number of documents deleted.
- Manages any errors thrown during connecting using the `catch` block.
- Ensures that the MongoDB client connection is closed using the `finally` block.

To execute the code, type following command in the Command Prompt:

```
node deleteop.js
```

The documents that matched with the specific criteria are deleted from the `books` collections in the `Library` database, as shown in Figure 9.18.

```
C:\Users\Linda\nodejs>node deleteop.js
Book deleted successfully
```

Figure 9.18: Console Displaying the Message of Document Deletion

Verify if the document with `Book_id: 2001` has been deleted by retrieving the existing documents in the `books` collection.

Type following command in the Command Prompt:

```
node readop.js
```

The document with `Book_id` as `2001` has been deleted from the `books` collection, as shown in Figure 9.19.

```
C:\Users\Linda\nodejs>node deleteop.js
Book deleted successfully

C:\Users\Linda\nodejs>node readop.js
Found 2 documents in the "books" collection:
[
  {
    _id: new ObjectId('65785deb18ee8801b5712490'),
    Book_id: 101,
    Book_name: 'The Secret of The House On The Hill',
    Author_name: 'Julia Harris',
    Price: '$10',
    Age_group: '15+',
    Book_type: 'Thriller'
  },
  {
    _id: new ObjectId('65785deb18ee8801b5712492'),
    Book_id: 3001,
    Book_name: 'When You Wish Upon A Star',
    Author_name: 'Ashlyn Clark',
    Price: '$18.99',
    Age_group: '18+',
    Book_type: 'Romance'
  }
]
```

Figure 9.19: Console Displaying the Existing Documents After Deletion

9.4 Building CRUD APIs in Node.js

API stands for Application Programming Interface. Let us build a basic Web API to perform the CRUD operations on the `books` collection of the `Library` MongoDB database using Node.js and Express.js environments.

Node.js and Express.js have already been installed. Now, complete following steps:

23. Open VS Code or Notepad and type the code given in Code Snippet 6.

Code Snippet 6:

```
const express = require('express');
const { MongoClient, ObjectId } = require('mongodb');
const app = express();
const port = 3000;
const url = 'mongodb://localhost:27017';
const databaseName = 'Library';
const collectionName = 'books';
app.use(express.json());
// Create - Add a new book
app.post('/books', (req, res) => {
  const newBook = req.body;
  MongoClient.connect(url)
    .then(client => {
      const db = client.db(databaseName);
      const collection = db.collection(collectionName);
      return collection.insertOne(newBook);
    })
    .then(result => {
      res.status(201).json(newBook);
    })
    .catch(error => {
      // Handle any other errors, and log detailed error
      // information
      console.error('Error during book insertion:', error);
      res.status(500).json({ error: 'Failed to add a new book'
    });
  });
});

// Read - Get all books
app.get('/books', (req, res) => {
  MongoClient.connect(url)
    .then(client => {
      const db = client.db(databaseName);
      const collection = db.collection(collectionName);
      return collection.find({}).toArray();
    })
    .then(books => {
      res.json(books);
    })
    .catch(error => {
      console.error('Error during book retrieval:', error);
      res.status(500).json({ error: 'Failed to retrieve books'
    });
  });
});
```

```

    })
    .then(books => {
        res.status(200).json(books); // Respond with the list of
books
    })
    .catch(error => {
        res.status(500).json({ error: 'Failed to retrieve books'
});
    });
});

// Read - Get a single book by Book_id
app.get('/books/:id', (req, res) => {
    const bookId = req.params.id;
    MongoClient.connect(url)
        .then(client => {
            const db = client.db(databaseName);
            const collection = db.collection(collectionName);
            return collection.findOne({ Book_id: parseInt(bookId)
});
        })
        .then(book => {
            if (!book) {
                console.error(`Book with Book_id ${bookId} not
found.`);
                res.status(404).json({ error: 'Book not found' });
            } else {
                res.status(200).json(book);
            }
        })
        .catch(error => {
            console.error('Error during book retrieval:', error);
            res.status(500).json({ error: 'Failed to retrieve the
book' });
        });
});

// Update - Modify a book by Book_id
app.put('/books/:id', (req, res) => {
    const bookId = req.params.id;
    const updatedBook = req.body;
    MongoClient.connect(url)
        .then(client => {
            const db = client.db(databaseName);
            const collection = db.collection(collectionName);
            return collection.updateOne({ Book_id: parseInt(bookId)
}, { $set: updatedBook });
        })
        .then(result => {

```



```

        if (result.matchedCount === 0) {
            res.status(404).json({ error: 'Book not found' });
        } else {
            res.status(200).json({ message: 'Book updated successfully' });
        }
    })
    .catch(error => {
        res.status(500).json({ error: 'Failed to update the book' });
    });
});

// Delete - Remove a book by Book_id
app.delete('/books/:id', (req, res) => {
    const bookId = req.params.id;
    MongoClient.connect(url)
        .then(client => {
            const db = client.db(databaseName);
            const collection = db.collection(collectionName);
            return collection.deleteOne({ Book_id: parseInt(bookId) });
        })
        .then(result => {
            if (result.deletedCount === 0) {
                res.status(404).json({ error: 'Book not found' });
            } else {
                res.status(200).json({ message: 'Book deleted successfully' });
            }
        })
        .catch(error => {
            res.status(500).json({ error: 'Failed to delete the book' });
        });
});

app.listen(port, () => {
    console.log(`Server is running on port ${port}`);
});

```

24. Save the code as `libraryapi.js`.

Table 9.2 describes the packages and the constants defined in Code Snippet 6.

Packages/Constants	Description
<pre>const express = require('express');</pre>	Imports the required packages, namely: <code>express</code> to create the API

Packages/Constants	Description
<pre>const { MongoClient, ObjectID } = require('mongodb');</pre>	• MongoClient to connect with the database • ObjectID to work with MongoDB ObjectIds
<pre>const app = express(); const port = 3000; const url = 'mongodb://localhost:27017'; const databaseName = 'Library'; const collectionName = 'books';</pre>	Creates an Express application instance named <code>app</code> on the specified port number to define the API routes and manage HTTP requests. Further, defines the MongoDB connection details, such as the <code>url</code> , <code>databaseName</code> , and <code>collectionName</code> .

Table 9.2: Packages and Constants in Code Snippet 6

Table 9.3 discusses the methods and properties used in Code Snippet 6 to perform the CRUD operations.

Methods/Properties	Description
<code>app.use(express.json())</code>	Parses incoming data requests in JSON format with the <code>express.json</code> middleware.
<code>app.listen()</code>	Binds and listens to the connections on the specified host and port.
<code>app.post()</code>	Routes the HTTP <code>post</code> requests to the specified path. Adds new books to the database.
<code>app.get()</code>	Routes the HTTP <code>get</code> requests to the specified path. Retrieves the book from the collection using the <code>book_id</code> parameter.
<code>app.put()</code>	Routes the HTTP <code>put</code> requests to the specified path. Modifies the details of the book using the <code>book_id</code> parameter.
<code>app.delete()</code>	Routes the HTTP <code>delete</code> requests to the specified path. Deletes the book using the <code>book_id</code> parameter.
<code>req.body</code>	Property of the Express.js <code>req</code> object that populates the parsed data.

Methods/Properties	Description
<code>req.params.id</code>	Property of the Express.js <code>req.params</code> object that holds the <code>id</code> value passed in the route parameter. Used for fetching, deleting, and updating book details.
<code>res.status()</code>	Sets the HTTP status for the response corresponding to the success or failure of the operation.

Table 9.3: Methods and Properties to Perform the CRUD Operations

To execute `libraryapi.js`, type following command in the Command Prompt:

```
node libraryapi.js
```

Find the Express server up and running. A message is logged to the console to indicate that the server is running on the specified port, as shown in Figure 9.20.

```
C:\Users\Linda\nodejs>node libraryapi.js
Server is running on port 3000
```

Figure 9.20: Console Displaying the Port Number of the Server

It is now possible to send HTTP requests to the specified API endpoints. Generally, the requests can be made using tools, such as `curl`, Postman, or Web browser.

Use `curl` to access the API endpoints and perform following tasks:

View all the books
Access a specific book
Add (Create) a new book
Read a book
Update a book
Delete a book

Type the corresponding commands in the Command Prompt to perform the respective tasks. Note that, before opening a new Command Prompt, confirm that the server is running on port 3000.

1. To view the books in the `books` collection, type:

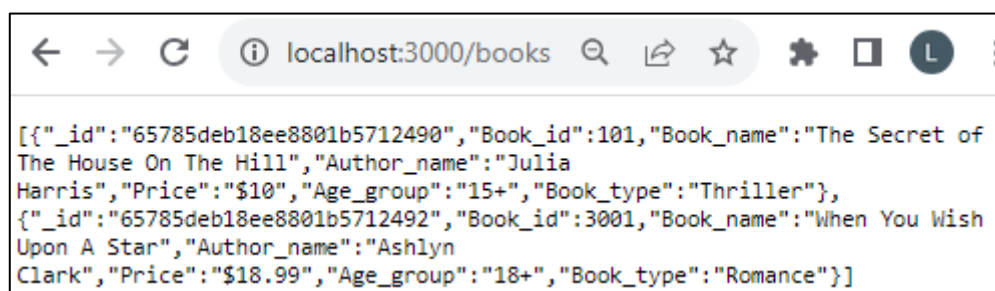
```
curl http://localhost:3000/books
```

Figure 9.21 displays the output of the execution of this command.

```
C:\Users\Linda\nodejs>curl http://localhost:3000/books
[{"_id":"65785deb18ee8801b5712490","Book_id":101,"Book_name":
"The Secret of The House On The Hill","Author_name":"Julia Ha
rris","Price":"$10","Age_group":"15+","Book_type":"Thriller"}
,{"_id":"65785deb18ee8801b5712492","Book_id":3001,"Book_name"
:"When You Wish Upon A Star","Author_name":"Ashlyn Clark","Pr
ice":"$18.99","Age_group":"18+","Book_type":"Romance"}]
```

Figure 9.21: Information of all the Books

Figure 9.22 shows all the documents displayed in the browser.



The screenshot shows a web browser window with the address bar set to `localhost:3000/books`. The main content area displays the same JSON array as in Figure 9.21, representing the data retrieved from the database.

Figure 9.22: Information of all Books in the Browser

2. To access details of a specific book by ID, for instance, 101, type:

```
curl http://localhost:3000/books/101
```

Figure 9.23 displays the output of the execution of this command.

```
C:\Users\Linda\nodejs>curl http://localhost:3000/books/101
{"_id":"65785deb18ee8801b5712490","Book_id":101,"Book_name":
"The Secret of The House On The Hill","Author_name":"Julia Har
ris","Price":"$10","Age_group":"15+","Book_type":"Thriller"}
```

Figure 9.23: Retrieval of the Specified Book Information

Figure 9.24 shows the document of the `Book_id` as 101 displayed in the browser.

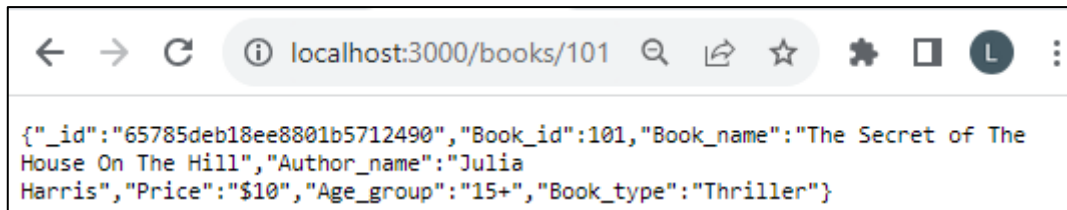


Figure 9.24: Document of Book ID 101

3. To add a book to the `books` collection, type:

```

curl -X POST -H "Content-Type: application/json" -d
'{"Book_id":1088, "Book_name": "Death and Me",
"Author_name": "Noah Wilson", "Price": "$16",
"Age_group":"16+", "Book_type":"Thriller"}'
http://localhost:3000/books

```

The command adds a book with following details:

- Book_id: 1088
- Book_name: Death and Me
- Author_name: Noah Wilson
- Price: \$16
- Age_group: 16+
- Book_type: Thriller

Figure 9.25 displays the output of the execution of this command.

```

C:\Users\Linda\nodejs>curl -X POST -H "Content-Type: application/json" -d '{"Book_id":1088, "Book_name": "Death and Me", "Author_name": "Noah Wilson", "Price": "$16", "Age_group":"16+", "Book_type":"Thriller"}' http://localhost:3000/books
{"Book_id":1088,"Book_name":"Death and Me","Author_name":"Noah Wilson","Price":"$16","Age_group":"16+","Book_type":"Thriller","_id":"657863b6c2894dab31d8b4db"}

```

Figure 9.25: Addition of a New Book

Figure 9.26 shows the documents in the `books` collection after the add operation displayed in the browser.

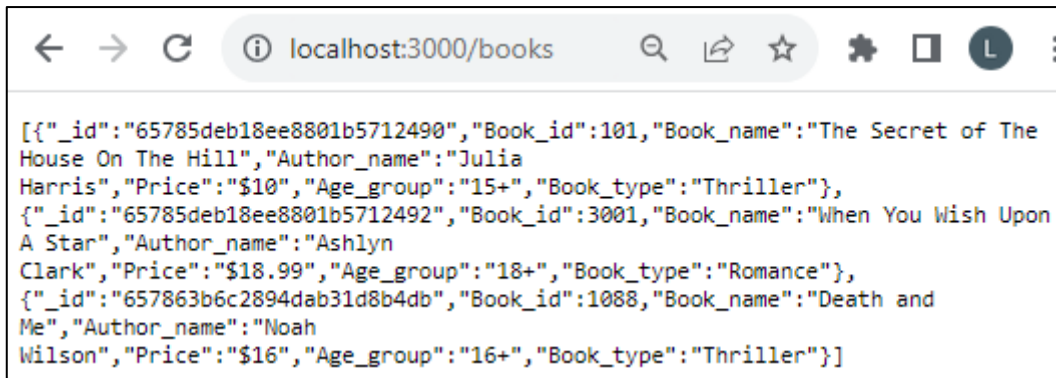


Figure 9.26: Documents after Add Operation

4. To update the information of a specific book by ID, type:

```
curl -X PUT -H "Content-Type: application/json" -d
"{\"Book_id\":1088, \"Book_name\": \"Death and Me\",
\"Author_name\": \"Noah Wilson\", \"Price\": \"$20\",
\"Age_group\":\"18+\", \"Book_type\":\"Thriller\"}"
http://localhost:3000/books/1088
```

The command updates the details of the book with `book_id: 1088`, as follows:

- Price: \$20
- Age_group: 18+

Figure 9.27 displays the output of the execution of this command.

```
C:\Users\Linda\nodejs>curl -X PUT -H "Content-Type: applicati
on/json" -d "{\"Book_id\":1088, \"Book_name\": \"Death and Me
\", \"Author_name\": \"Noah Wilson\", \"Price\": \"$20\", \"A
ge_group\":\"18+\", \"Book_type\":\"Thriller\"}" http://local
host:3000/books/1088
{"message":"Book updated successfully"}
```

Figure 9.27: Retrieval of the Specified Book Information

Figure 9.28 shows the updated document displayed in the browser.

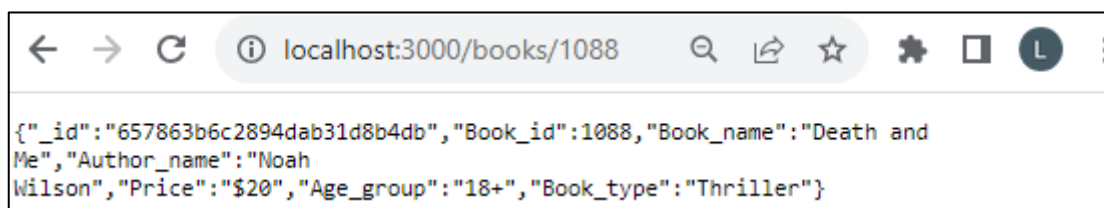


Figure 9.28: Updated Document

5. To delete a specific book by ID, type:

```
curl -X DELETE http://localhost:3000/books/101
```

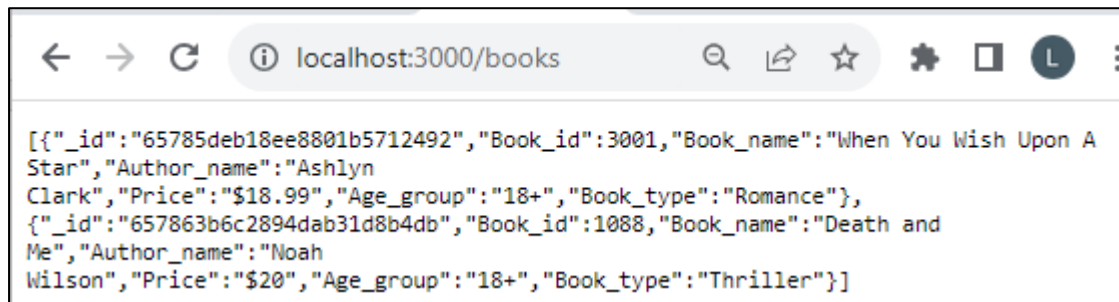
The command uses the `DELETE` request to delete an existing book with `book_id: 101`.

Figure 9.29 displays the output of the execution of this command.

```
C:\Users\Linda\nodejs>curl -X DELETE http://localhost:3000/books/101
{"message":"Book deleted successfully"}
```

Figure 9.29: Deletion of the Book

Figure 9.30 shows the documents in the `books` collection after the delete operation displayed in the browser.



```
[{"_id":"65785deb18ee8801b5712492","Book_id":3001,"Book_name":"When You Wish Upon A Star","Author_name":"Ashlyn Clark","Price":"$18.99","Age_group":"18+","Book_type":"Romance"}, {"_id":"657863b6c2894dab31d8b4db","Book_id":1088,"Book_name":"Death and Me","Author_name":"Noah Wilson","Price":"$20","Age_group":"18+","Book_type":"Thriller"}]
```

Figure 9.30: Documents after Delete Operation

9.5 Summary

- Using Node.js developers can connect with both relational and non-relational databases.
- MongoDB is a non-relational database.
- Express is a Node.js Web framework that can be used to create Web applications and APIs.
- The MongoDB Node.js Driver is the communication tool that is internally used for communication between Node.js and MongoDB.
- Libraries, such as `mongodb` and `express` can be imported into the code with the `require` keyword.
- Command line tools, such as `curl` help in transferring data over network protocols, such as HTTP.

Test Your Knowledge

1. What type of database does Node.js support?
 - a) MySQL
 - b) Oracle
 - c) MongoDB
 - d) All of these
2. In which format does Node.js exchange data?
 - a) CSV
 - b) JSON
 - c) XML
 - d) Text
3. You have started working on a MongoDB database. Which is the correct way to import the MongoClient class?
 - a) `const { mongodb } = require('mongodb')`
 - b) `const { MongoClient }`
 - c) `const { mongodb } = require('MongoClient')`
 - d) `const { MongoClient } = require('mongodb')`
4. Which tool is used to access API by sending HTTP requests to the endpoint?
 - a) Express
 - b) Init
 - c) Curl
 - d) mongodb
5. Which operation will you use to add a new record or document?
 - a) Create
 - b) Read
 - c) Update
 - d) Delete

Answers to Test Your Knowledge

1	d
2	b
3	d
4	c
5	a

Try It Yourself

1. Create a simple Web application for the art gallery and perform CRUD operations on the `GalleryOne` in the `Art` database. Table 9.4 shows the data for this collection.

ID	Artist Name	Art Type	Price
1	Annie	Painting	120\$
2	Jack	Sculpture	100\$
3	Paul	Pop art	130\$
4	Harry	Abstract art	190\$

Table 9.4: Data for `GalleryOne`

- a) Import the necessary packages.
- b) Create a MongoDB database named, `Art`.
- c) Add a collection named `GalleryOne`.
- d) Insert the four documents given in Table 9.4 into this collection.
- e) View the inserted documents.
- f) Update the `Price` and `Artist Name` in the document with `ID` as 4 to 210\$ and `Grace` respectively.
- g) Delete the document with `ID` as 3.
- h) Display the results using `curl` as well as a Web browser.